

js笔记

好的，根据您提供的两份PPT文档（第3章 数组 和 第4章 函数），我为您梳理了应对考试所需的核心知识点。这份梳理将重点、难点和易错点进行了归纳，方便您复习。

第3章 数组 知识点梳理

一、数组基础概念

1. **定义**：用于存储一组相关联数据的集合。
2. **组成**：由**元素**组成，每个元素包含**索引（下标）**（下标，从0开始）和**值**（可以是任意数据类型）。
3. **特点**：数组是复杂数据类型（对象）。

二、数组的创建

1. **数组字面量**：`var arr = [元素1, 元素2, ...];`（推荐，最常用）
 - 允许有空位：`[1, , 3]`
2. **new Array()**：`var arr = new Array(元素1, 元素2, ...);`
 - **注意注意****：`new Array(3)` 表示创建一个长度为3的空数组，而非 `[3]`。

三、数组的基本操作（核心考点）

1. **获取长度**：`数组名.length`（空位会计入长度）
2. **修改长度**：`数组名.length = 数字`
 - 新长度 > 原长度：末尾补空位。
 - 新长度 < 原长度：从末尾删除多余元素。
3. **访问元素**：`数组名[索引]`
 - 索引超出范围，返回 `undefined`。
4. **遍历数组**：使用 `for` 循环。

JavaScript

```
1 for (var i = 0; i < arr.length; i++) {  
2     console.log(arr[i]);  
3 }
```

5. 添加/修改元素：数组名[索引] = 值

- - 索引 $\geq$ 数组长度 $\rightarrow$ 添加元素。
- 索引 $<$ 数组长度 $\rightarrow$ 修改元素。

6. 删除元素：delete 数组名[索引]

-注意：删除后，该位置变为空位，数组长度不变。

7. 筛选元素：结合 for 循环和 if 条件判断，将符合条件的元素放入新数组。

8.

- a. 反转元素顺序：从最后一个元素遍历到第一个元素，依次放入新数组。

四、数组元素排序（重点）

1. 冒泡排序

- 原理：比较相邻元素，大的往后（或往前）冒泡。
- 轮数：arr.length - 1
- 每轮比较次数：arr.length - 当前轮数
- 口诀：N个数字来排序，外层循环N-1，内层循环N-1-i。

2. 插入排序

- 原理：将数组分为“有序”和“无序”两部分，每次从无序部分取第一个元素，插入到有序部分的正确位置。

五、二维数组

1. 创建：var arr = [[1,2], [3,4], [5,6]];

2. 访问：数组名[行索引]（第一个是行索引，第二个是列索引）

3. 遍历：使用嵌套 for 循环。

JavaScript

```
1 for (var i = 0; i < arr.length; i++) {  
2     for (var j = 0; j < arr[i].length; j++) {  
3         console.log(arr[i][j]);  
4     }  
5 }
```

4. **应用**：矩阵转置 (`res[i][j] = arr[j][i]`)。

易错点

1. **类型**：`typeof []` 结果是 `"object"`。
2. **字符串索引**：`arr["fruit"] = "apple"` 是在给数组对象添加属性，而非数组元素。
3. **引用传递**：数组是对象，赋值给新变量时，新变量指向同一个数组，修改一个会影响另一个。

第4章 函数 知识点梳理

一、函数基础

1. **定义**：实现特定功能的代码块，提高复用性。
2. **分类**：内置函数 **定义函数**：

JavaScript

```
1 function 函数名(形参1, 形参2) {  
2     // 函数体  
3     return 返回值;  
4 }
```

3. **调用函数**：`函数名(实参1, 实参2);`
4. **参数**：
 - **形参**：定义时的参数。
 - **实参**：调用时传入的参数。
 - **数量不匹配**：实参多则忽略；实参少则对应形参为 `undefined`。
 - **arguments 对象**：函数内置对象，用于接收所有实参（即使没有定义形参）。
5. **返回值**：
 - 使用 `return` 返回结果。
 - 没有 `return` 或 `return` 后无值，则返回 `undefined`。
 - `return` 会立即终止函数。

二、函数进阶（重点）

1. 函数表达式：`：

JavaScript

```
1 var 变量名 = function(形参) { ... };  
2 // 调用：变量名(实参);
```

2. 匿名函数：没有名字的函数。

- 常用于函数表达式、事件处理、自调用。
- **匿名函数自调用**：(function() { ... })();

3. 回调函数：将一个函数A作为参数传递给另一个函数B，并在B内部调用A，则A是回调函数。

4. 递归函数：函数内部调用自身。

- **关键**：必须有**终止条件**，否则会死循环。
- **应用**：计算阶乘、遍历多维数组。

5. 箭头函数：fn(n) = n * fn(n-1)

6. 箭头函数 (ES6)：

- (参数) => { 函数体 }
- **简化写法**：
 - 只有一个参数可省略括号：x => x * 2
 - 函数体只有一条 return 语句可省略大括号和 return：(a, b) => a + b

三、作用域

1. **全局变量**：在函数外声明，或函数内**省略 var** 声明的变量。任何地方都可访问。
2. **局部变量**：在函数内使用 var 声明的变量。只能在函数内访问。
3. **块级变量**：使用 let 或 const 声明的变量，只在 {} 内有效。
4. **作用域链**：当访问变量时，会先在当前作用域查找，找不到则向上一级查找，直到全局作用域。这种链式关系称为作用域链。

四、闭包函数（难点）

1. **定义**：一个函数（内层函数）可以访问其外部函数作用域中的变量。
2. **作用**：

- 在函数外部读取函数内部的局部变量。
- 让变量的值始终保持在内存中（如统计函数调用次数）。

3. 实现：在一个函数内部返回另一个函数。

JavaScript

```
1 function outer() {  
2     var count = 0;  
3     return function inner() {  
4         count++;  
5         return count;  
6     }  
7 }  
8 var fn = outer();  
9 fn(); // 1  
10 fn(); // 2
```

五、预解析（变量提升）

1. **机制**：JavaScript引擎在执行代码前，会先将 **var** 声明的变量和 **function** 定义的函数提升到当前作用域的顶部。
2. **变量提升**：只提升声明，不提升赋值。所以先使用后声明，值为 **undefined**。
3. **函数提升**：整个函数定义被提升。所以可以在定义之前调用函数。
4. **注意**：函数表达式不会被提升（因为它是 **var** 变量提升，只提升变量名，值为 **undefined**，无法作为函数调用）。

考试复习建议：

- **数组**：重点练习冒泡排序和插入排序的手写代码逻辑，以及二维数组的遍历。
- **函数**：重点理解回调函数、递归函数（阶乘案例）和闭包函数（统计次数案例）的执行过程。
- **易错**：**delete delete** 数组元素不改变长度；**arguments`** 的使用；函数表达式与函数声明的预解析区别；闭包的内存占用问题。

祝您考试顺利！